TASK 3

1. Load a CSV dataset and display its shape, columns, and first 10 records.

```
import pandas as pd
import numpy as np
df = pd.read_csv('Teen_Mental_Health.csv')
```

```
print("Shape of Dataset\n", df.shape)
print("\nColumns :\n", df.columns)
print("\nFirst 10 records\n", df.head(10))
```

```
Shape of Dataset
 (1200, 16)

Columns :
 Index(['age', 'gender', 'daily_social_media_hours', 'platform_usage',
       'sleep_hours', 'screen_time_before_sleep', 'academic_performance',
       'physical_activity', 'social_interaction_level', 'stress_level',
       'anxiety_level', 'addiction_level', 'depression_label',
       'mental_health_risk_score', 'sleep_quality', 'digital_wellbeing_flag'],
      dtype='object')

First 10 records
    age  gender  daily_social_media_hours platform_usage  sleep_hours  \
0   14    male                       7.9       Facebook          7.4
1   19  female                       1.9         TikTok          8.0
2   17  female                       1.3      Instagram          7.6
3   15    male                       7.4        YouTube          6.9
4   15  female                       4.7  All Platforms          4.9
5   19  female                       7.4  All Platforms          4.4
6   18  female                       2.5      Instagram          6.4
7   16    male                       4.0  All Platforms          4.2
8   19  female                       3.3        YouTube          5.0
9   15    male                       1.9         TikTok          4.9

   screen_time_before_sleep  academic_performance  physical_activity  \
0                       2.9                  3.01                1.5
1                       2.9                  3.22                0.8
2                       0.5                  3.92                0.0
3                       1.6                  3.48                0.8
4                       3.0                  2.37                1.4
5                       2.4                  2.63                0.6
6                       2.4                  2.63                0.7
7                       0.5                  2.40                1.3
8                       2.1                  2.04                0.9
9                       1.5                  3.77                1.1
```

	social_interaction_level	stress_level	anxiety_level	addiction_level	\
0	low	2	2	1	
1	high	8	1	10	
2	high	2	4	2	
3	medium	1	7	9	
4	medium	3	5	2	
5	high	3	5	7	
6	low	2	2	5	
7	low	6	10	5	
8	high	1	10	9	
9	high	1	1	4	

	depression_label	mental_health_risk_score	sleep_quality	\
0	0	5	Fair	
1	0	19	Good	
2	0	8	Fair	
3	0	17	Fair	
4	0	10	Poor	
5	0	15	Poor	
6	0	9	Fair	
7	0	21	Poor	
8	0	20	Poor	
9	0	6	Poor	

	digital_wellbeing_flag
0	At Risk
1	Moderate
2	Healthy
3	Moderate
4	Moderate
5	At Risk
6	Moderate
7	Moderate
8	Moderate
9	Healthy

2. Identify and handle missing values using mean and mode.

```
print(df.isnull().sum())
num_cols = df.select_dtypes(include=np.number).columns
cat_cols = df.select_dtypes(include='object').columns
for col in num_cols:
    df[col].fillna(df[col].mean(), inplace=True)
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
print("\nMissing values after handling:")
print(df.isnull().sum())
```

```
age                         0
gender                      0
daily_social_media_hours    0
platform_usage              0
sleep_hours                 0
screen_time_before_sleep    0
academic_performance        0
physical_activity           0
social_interaction_level    0
stress_level                0
anxiety_level               0
addiction_level             0
depression_label            0
mental_health_risk_score    0
sleep_quality               0
digital_wellbeing_flag      0
dtype: int64

Missing values after handling:
age                         0
gender                      0
daily_social_media_hours    0
platform_usage              0
sleep_hours                 0
screen_time_before_sleep    0
academic_performance        0
physical_activity           0
social_interaction_level    0
stress_level                0
anxiety_level               0
addiction_level             0
depression_label            0
mental_health_risk_score    0
sleep_quality               0
digital_wellbeing_flag      0
dtype: int64
```

3. Normalize and standardize numerical features in a dataset.

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler

print("Normalization")
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(df[num_cols])
print(normalized_data[:5])

print("Standardization")
std_scaler = StandardScaler()
standardized_data = std_scaler.fit_transform(df[num_cols])
print(standardized_data[:5])
```

```
Normalization
[[0.16666667 0.98571429 0.68       0.96       0.505      0.75
  0.11111111 0.11111111 0.         0.         0.07407407]
 [1.         0.12857143 0.8        0.96       0.61       0.4
  0.77777778 0.         1.         0.         0.59259259]
 [0.66666667 0.04285714 0.72       0.         0.96       0.
  0.11111111 0.33333333 0.11111111 0.         0.18518519]
 [0.33333333 0.91428571 0.58       0.44       0.74       0.4
  0.         0.66666667 0.88888889 0.         0.51851852]
 [0.33333333 0.52857143 0.18       1.         0.185      0.7
  0.22222222 0.44444444 0.11111111 0.         0.25925926]]
Standardization
[[-0.95409872  1.6578328   0.65917703  1.61882976  0.03402616  0.83427567
  -1.18736693 -1.27233544 -1.613389   -0.16284469 -2.31283468]
 [ 1.51979597 -1.29964891  1.07524389  1.61882976  0.39828246 -0.36859347
   0.88011599 -1.62219852  1.56744364 -0.16284469  0.46713403]
 [ 0.5302381  -1.59539708  0.79786598 -1.73143592  1.61247011 -1.74330107
  -1.18736693 -0.57260926 -1.25996315 -0.16284469 -1.7171271 ]
 [-0.45931978  1.41137599  0.31245465 -0.19589748  0.84926644 -0.36859347
  -1.53194742  0.47698001  1.21401779 -0.16284469  0.06999564]
 [-0.45931978  0.08050922 -1.07443487  1.75842417 -1.07608826  0.66243722
  -0.84278644 -0.22274617 -1.25996315 -0.16284469 -1.31998871]]
```

4. Split a dataset into training and testing sets.

```
from sklearn.model_selection import train_test_split

X = df[num_cols]
y = df['academic_performance']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

```
Training Shape: (960, 11)
Testing Shape: (240, 11)
```

5. Build a Linear Regression model to predict house prices.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.DataFrame({
    'math_score': [35, 78, 45, 90, 55, 60, 30, 88, 70, 40],
    'science_score': [40, 80, 50, 92, 58, 65, 28, 85, 75, 38],
    'english_score': [50, 70, 55, 95, 60, 62, 35, 90, 72, 45]
})

# average performance
df['academic_performance'] = df.mean(axis=1)

median_score = df['academic_performance'].median()

df['result'] = df['academic_performance'].apply(lambda x: 1 if x >= median_score else 0)

X = df[['math_score','science_score','english_score']]
y = df['result']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Predictions:", pred)
```

```
Accuracy: 1.0
Predictions: [1 1]
```

6. Build a Multiple Linear Regression model for salary prediction.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

salary_df = pd.DataFrame({
    'experience': [1,2,3,4,5,6,7,8,9,10],
    'hours_worked': [30,35,40,45,50,55,60,65,70,75],
    'salary': [20000,25000,30000,35000,40000,45000,52000,58000,65000,72000]
})

X = salary_df[['experience', 'hours_worked']]
y = salary_df['salary']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

print(model.predict(X_test))
```

```
[64034.48275862 23965.51724138]
```

7. Create a Logistic Regression model to predict student pass/fail outcomes.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

house_df = pd.DataFrame({
    'area': [1000,1200,1500,1800,2000,2200,2500,2800,3000,3500],
    'bedrooms': [2,2,3,3,4,4,4,5,5,6],
    'price': [200000,250000,300000,350000,400000,450000,500000,550000,600000,700000]
})

X = house_df[['area']]
y = house_df['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

print(model.predict(X_test))
```

```
[598282.94687408 239286.7625477 ]
```

8. Train a Decision Tree classifier for loan approval prediction.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

loan_df = pd.DataFrame({
    'income': [25,40,50,60,80,35,45,70,90,100],
    'credit_score': [600,650,700,750,800,620,680,720,760,820],
    'loan_status': [0,0,1,1,1,0,1,1,1,1]
})

X = loan_df[['income','credit_score']]
y = loan_df['loan_status']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Predictions:", pred)
```

```
Predictions: [1 0]
```

9. Implement K-Nearest Neighbors (KNN) for classification.

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)

pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
```

```
Accuracy: 0.5
```

10. Build a Random Forest model for customer churn prediction.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train, y_train)

pred = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
```

```
Accuracy: 1.0
```

11. Calculate Accuracy, Precision, Recall, and F1 Score for a model.

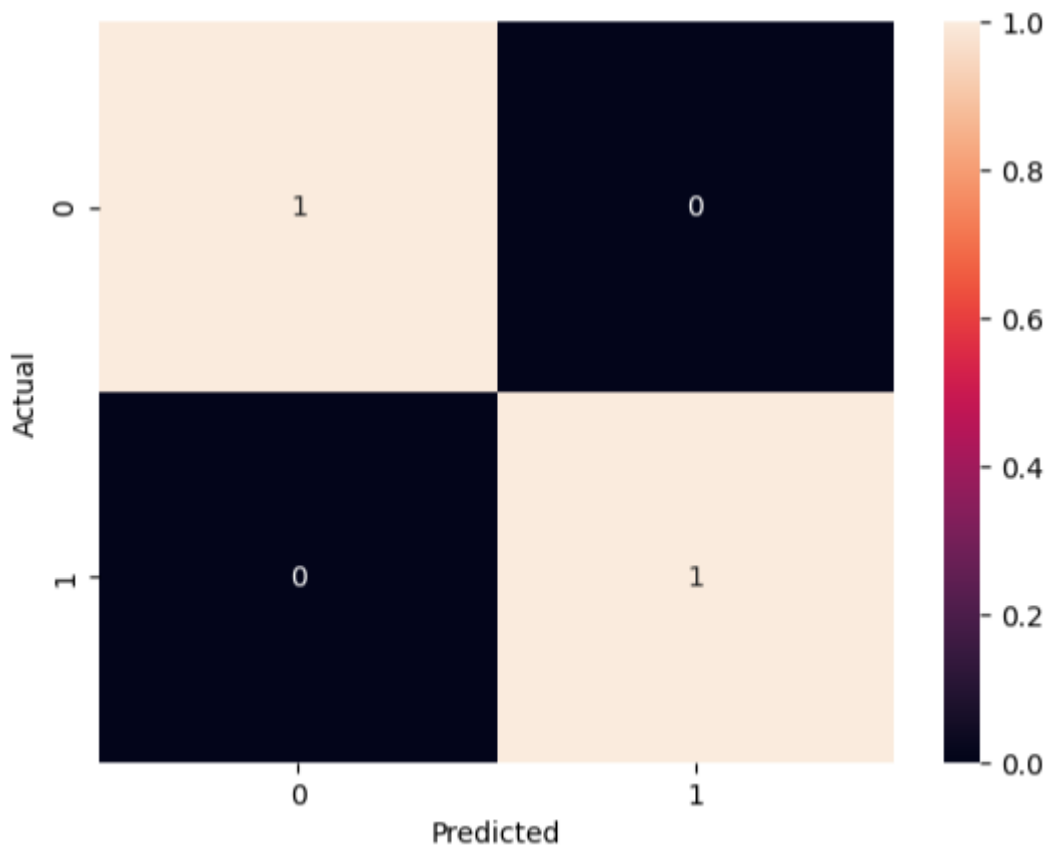
```
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred))
print("Recall:", recall_score(y_test, pred))
print("F1 Score:", f1_score(y_test, pred))
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

12. Generate and interpret a Confusion Matrix.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, pred)

print(cm)

sns.heatmap(cm, annot=True, fmt='d')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
[[1 0]
 [0 1]]
```



13. Identify important features contributing to model predictions.

```
importance = rf.feature_importances_

feature_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
})

print(feature_df.sort_values(
    by='Importance',
    ascending=False
))
```

```
        Feature  Importance
0        income    0.505376
1  credit_score    0.494624
```

14. Apply K-Fold Cross Validation to evaluate model performance.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(rf, X, y, cv=5)

print("Scores:", scores)

print("Average Accuracy:", scores.mean())
```

```
C:\Users\Jane Brenna Christy\anaconda3\Lib\site-packa
ast populated class in y has only 3 members, which is
  warnings.warn(
Scores: [1.  1.  1.  0.5 1. ]
Average Accuracy: 0.9
```

15. Perform customer segmentation using K-Means clustering.

```
drop_cols = ['academic_performance']

num_cols = df.select_dtypes(include=np.number).columns.drop(
    [col for col in drop_cols if col in df.columns]
)

kmeans = KMeans(n_clusters=3, random_state=42)

clusters = kmeans.fit_predict(df[num_cols])

df['Cluster'] = clusters
print(clusters)
```

```
[2 0 1 0 1]
```

16. Clean text by converting to lowercase and removing punctuation.

```
import string

text = "Hello! This is NLP Example."

text = text.lower()

text = text.translate(
    str.maketrans('', '', string.punctuation)
)

print(text)
```

```
hello this is nlp example
```

17. Tokenize a paragraph into sentences and words.

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

nltk.download('punkt')

paragraph = "Natural Language Processing is interesting. It is used in AI."

sentences = sent_tokenize(paragraph)

words = word_tokenize(paragraph)

print(sentences)

print(words)
```

```
[nltk_data] Downloading package punkt to C:\Users\Jane Brenna
[nltk_data]     Christy\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
['Natural Language Processing is interesting.', 'It is used in AI.']
['Natural', 'Language', 'Processing', 'is', 'interesting', '.', 'It', 'is', 'used', 'in', 'AI', '.']
```

18. Remove stopwords from a text dataset.

```
from nltk.corpus import stopwords

nltk.download('stopwords')

text = "This is an example sentence for stopword removal"

stop_words = set(stopwords.words('english'))

words = word_tokenize(text)

filtered_words = [
    word for word in words
    if word.lower() not in stop_words
]

print(filtered_words)
```

```
['example', 'sentence', 'stopword', 'removal']
```

19. Count word frequency in a given text corpus.

```
text = "AI AI machine learning learning NLP"

words = word_tokenize(text.lower())

freq = {}

for word in words:
    freq[word] = freq.get(word, 0) + 1

print(freq)
```

```
{'ai': 2, 'machine': 1, 'learning': 2, 'nlp': 1}
```

20. Apply stemming to reduce words to root forms.

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = ['running', 'playing', 'studies']

stemmed = [
    stemmer.stem(word)
    for word in words
]

print(stemmed)
```

```
['run', 'play', 'studi']
```

21. Apply lemmatization to convert words into dictionary forms.

```
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()

words = ['running', 'better', 'studies']

lemmatized = [
    lemmatizer.lemmatize(word)
    for word in words
]

print(lemmatized)
```

```
['running', 'better', 'study']
```

22. Convert text into vectors using the Bag of Words approach.

```
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    'I love machine learning',
    'Machine learning is powerful'
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())

print(X.toarray())
```

```
['is' 'learning' 'love' 'machine' 'powerful']
[[0 1 1 1 0]
 [1 1 0 1 1]]
```

23. Generate TF-IDF vectors for document analysis.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())

print(X.toarray())
```

```
['is' 'learning' 'love' 'machine' 'powerful']
[[0.         0.50154891 0.70490949 0.50154891 0.        ]
 [0.57615236 0.40993715 0.         0.40993715 0.57615236]]
```

24. Build a sentiment analysis model to classify text sentiment.

```
from sklearn.naive_bayes import MultinomialNB

reviews = [
    'I love this product',
    'This is terrible',
    'Amazing experience',
    'Worst service ever'
]

labels = [1,0,1,0]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(reviews)

y = labels

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2
)

model = MultinomialNB()

model.fit(X_train, y_train)

pred = model.predict(X_test)

print(pred)
```

```
[0]
```

25. Create a spam detection model using NLP and machine learning

```
messages = [
    'Congratulations! You won a prize',
    'Meeting at 5 PM',
    'Claim your free reward now',
    'Project submission tomorrow'
]

labels = [1,0,1,0]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(messages)

y = labels

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2
)

model = MultinomialNB()

model.fit(X_train, y_train)

pred = model.predict(X_test)

print(pred)
```

```
[0]
```